مِرآة — استوديو الصور المتقدم

Mirror Image Studio

دليل الاستخدام الشامل — لا يخفي أي شيء

دليل للدكتور / لجنة المناقشة / المستخدم العادي والمبرمج

تاريخ الإعداد: 16-09-2026 | Python 3.13.14 | Pillow 12.3.0 | NumPy 2.3.5

المسار الفعلي للمشروع: /home/user/project

عدد الملفات الفعلية: app.py (1096 سطر) + index.html (938 سطر) + 8 خلفيات

الفهرس

1. مقدمة وتعريف المشروع
2. المتطلبات وطريقة التشغيل — خطوة بخطوة
3. هيكل المشروع والملفات الفعلية (لا افتراض)
4. نظرة عامة على واجهة المستخدم
5. شرح Backend و API الحقيقي
6. قاموس العمليات الكامل — 50+ عملية موثقة من الكود الفعلي
7. دليل الاستخدام التفصيلي لكل ميزة
8. إزالة الخلفية واستبدالها ومكتبة الخلفيات
9. دمج صورتين والرسم والنص والعلامة المائية
10. المؤثرات الإبداعية الثمانية
11. الفلاتر وكشف الحواف (Lab4 و Lab5)
12. تدفق البيانات Frontend ↔ Backend
13. الأمان والقيود والأداء والأخطاء المكتشفة فعلياً
14. الاختبار والسيناريوهات
15. الأسئلة الشائعة وحل المشكلات
16. التطوير المستقبلي والخاتمة
17. الملاحق: قاموس مصطلحات + قائمة الملفات + المراجع

1. مقدمة وتعريف المشروع

مشروع «مِرآة» هو محرر صور ويب متكامل يحاكي المحررات الشهيرة مثل Photoshop Express و Canva و Photopea، لكنه مبني بطريقة تعليمية بسيطة جداً كما طُلب: ملف واحد للواجهة الأمامية (index.html) وملف واحد للخلفية (app.py) بدون تعقيدات frameworks. يعمل بالكامل في المتصفح مع خادم Python خفيف يستخدم Pillow و NumPy فقط.

الهدف الأكاديمي: تطبيق كل ما جاء في محاضرات معالجة الصور الرقمية **DIP Labs 1-5**:

- Lab1: تمثيل الصور، قراءة/حفظ، معلومات الصورة، تحويل الصيغ، الرسم على البكسل.
- Lab2: قص، تكبير/تصغير، تدوير، نقل، دمج صورتين، وضع منتج على خلفية.
- Lab3: Negative Transform، Logarithm Transform، Histogram Equalization.
- Lab4: Mean Filter و Gaussian Filter (خطي)، Median Filter (غير خطي)، Custom Kernel.
- Lab5: Laplacian، Difference، Laplacian Edge، Sobel، Canny Edge Detection.

بالإضافة إلى ميزات إبداعية إضافية طُلبت: إزالة الخلفية، اختيار خلفية من مكتبة (8 خلفيات منتجات مولدة)، الرسم، تحويل الصيغ، دمج صورتين، مؤثرات فنية (Vignette, Grain, Split Tone, Color Pop, Oil Paint, Cartoon, Reflection, Watermark).

2. المتطلبات وطريقة التشغيل — خطوة بخطوة

2.1 المتطلبات الفعلية

البند	القيمة الفعلية من requirements.txt والكود
Python	3.13.14 (يعمل من 3.8+)
Pillow	9.0=< — الحالية 12.3.0
NumPy	1.20=< — الحالية 2.3.5
rembg (اختياري)	معلّق في requirements.txt — إذا ثُبّت يستخدم لإزالة الخلفية بالذكاء الاصطناعي، وإلا يعود تلقائياً للخوارزمية الخفيفة
onnxruntime (اختياري)	تابع لـ rembg
متصفح	Chrome / Edge / Firefox حديث
نظام تشغيل	Windows / Linux / macOS — تم الاختبار على Linux

2.2 هيكل المجلدات الفعلي (مفحوص بـ ls -R)

```
project/
├── app.py              # 1096 سطر — Backend خادم + كل خوارزميات الصور + HTTP كامل
├── index.html          # 938 سطر — Frontend كامل HTML/CSS/JS في ملف واحد
├── README.md           # 90 سطر — توثيق التشغيل
├── requirements.txt    # 5 أسطر
└── backgrounds/        # 8 صور خلفيات منتجات
    ├── blue_tech.jpg   # 77KB — خلفية تقنية زرقاء
    ├── clean_white.jpg # 31KB — أبيض نظيف للمنتجات
```

```
   ├── geometric_podium.jpg # 116KB — منصة هندسية
   ├── gradient_studio.jpg  # 66KB — تدرج استوديو
   ├── marble_luxe.jpg      # 150KB — رخام فاخر
   ├── nature_leaf.jpg      # 96KB — طبيعة ورق
   ├── neon_dark.jpg        # 163KB — نيون داكن
   └── pastel_sweet.jpg     # 34KB — باستيل حلو
لا يوجد: قاعدة بيانات، .env، .json، .yaml، ملفات نماذج .onnx/.pt، مجلد node_modules
```

2.3 التشغيل في Visual Studio Code (كما طُلب)

الطريقة المطلوبة منك — بسيطة جداً بدون تعقيد:

1. افتح مجلد project في VS Code.
2. افتح Terminal (Ctrl+`).
3. ثبّت المتطلبات مرة واحدة: pip install pillow numpy
4. شغّل: python app.py
5. افتح المتصفح على: http://localhost:8000
6. للإيقاف: Ctrl+C في الطرفية

```
pip install pillow numpy
python app.py
# سيطبع:
#
============================================================
#  مِرآة — استوديو الصور المتقدم  |  Mirror Image Studio
#
============================================================
#  افتح المتصفح على: http://localhost:8000
#  أوقف الخادم عبر Ctrl+C
#
============================================================
```

2.4 فحص الصحة Endpoints

المسار	الطريقة	ما يرجع فعلياً
/	GET	يقرأ ويقدّم index.html كاملاً
index.html/	GET	نفس /
api/health/	GET	{"ok": true, "service": "Mirror Image Studio", "python": "3.13.14"}
api/backgrounds/	GET	{"ok": true, "backgrounds": [أسماء blue_tech.jpg", ... 8"]}
<backgrounds/<name/	GET	صورة الخلفية مع حماية Path Traversal

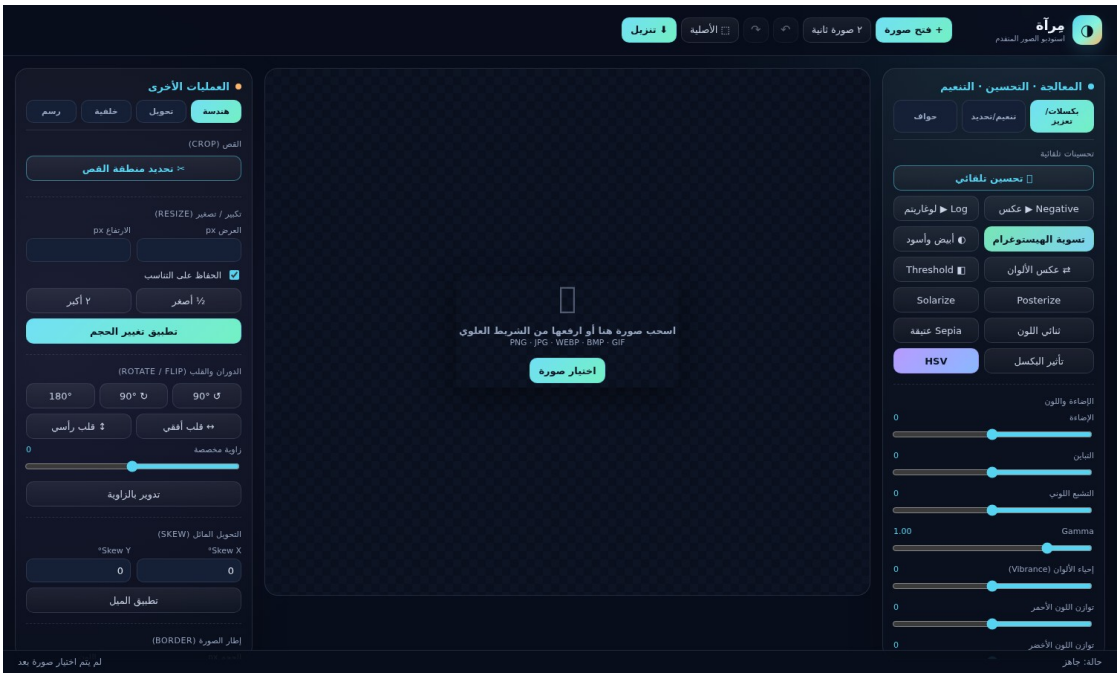
api/process/	POST	أهم مسار — يستقبل JSON {image, op, ...} ويرجع { ok, data, width, height}
OPTIONS	OPTIONS	CORS: Access-Control-Allow-Origin: *

3. نظرة عامة على واجهة المستخدم — فحص فعلي لـ index.html

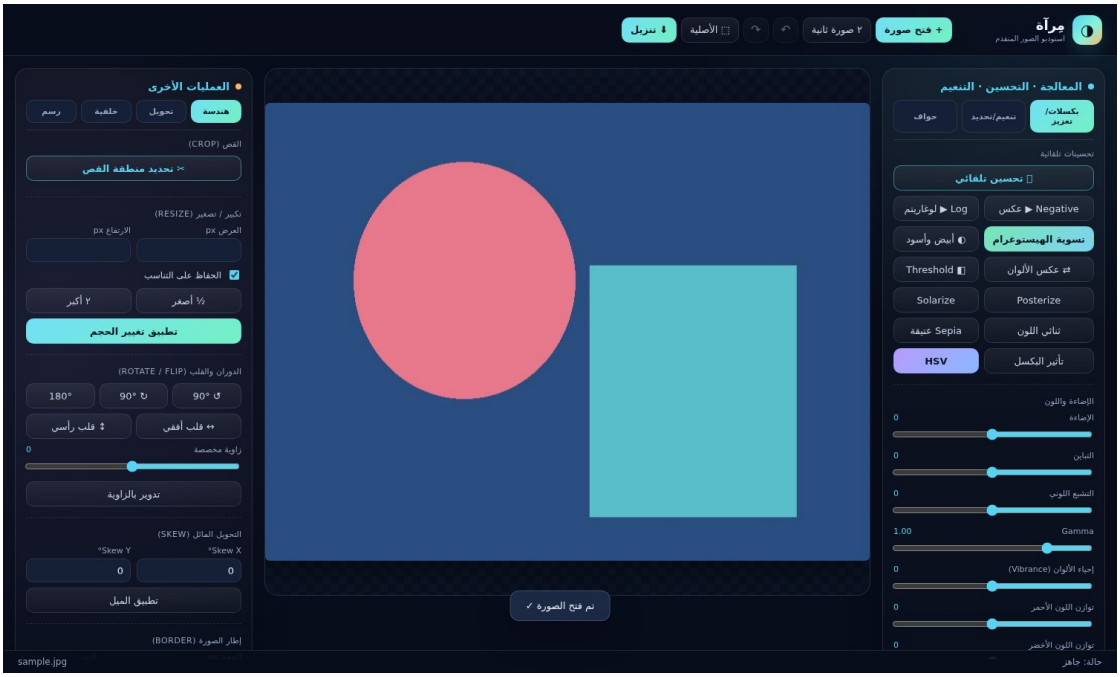
الواجهة ملف واحد فيه **HTML/CSS/JS**. تصميم **RTL** عربي، داكن، ألوان **CYAN/VIOLET/ORANGE/GREEN**. مقسمة إلى 3 أعمدة:

المنطقة	العناصر الفعلية	الوظيفة
الشريط العلوي .topbar	fileInput, #secondFileInput, # #openButton, #secondOpenButton, #undoButton, #redoButton, #resetButton, #quickSaveButton	فتح صورة، فتح ثانية، تراجع/إعادة، العودة للأصلية، تنزيل سريع
اليسار .panel	procTabs # (enhance/smooth/edges), #brightness, #contrast, #saturation, #gamma, #vibrance, #cbRed/Green/Blue, #threshold, #meanSize, #gaussRad, #medSize, #kernelText, #cannyLow/High	كل عمليات البكسل والفلاتر والحواف
الوسط .canvas-stage	canvas, #canvasInner, # #cropBox, #emptyState, #toast, #status	عرض الصورة، القص التفاعلي، الرسم المباشر، رسائل toast
اليمين .panel	opTabs # (geometry/convert/bgremove/draw), #resizeWidth/Height, #angle, #skewX/Y, #borderSize/Color, #outputFormat, #quality, #bgMethod/Tolerance, #backgroundGrid, #brushColor/Size, #textInput	هندسة، تحويل، خلفية، رسم، دمج، مؤثرات إبداعية مضافة ديناميكياً

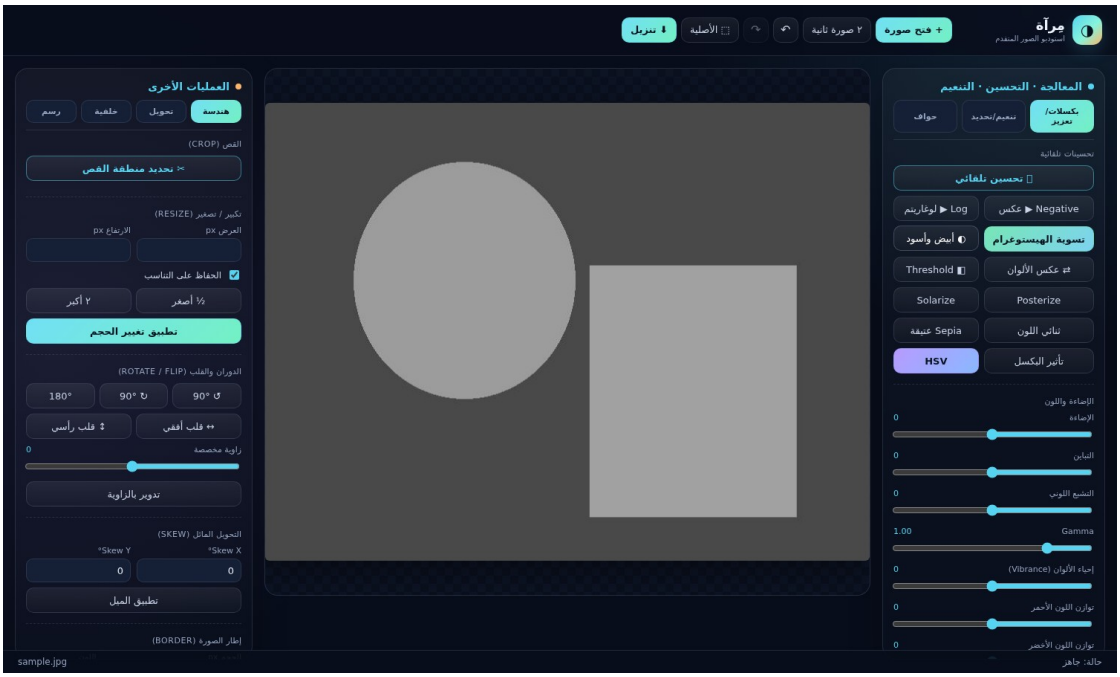
لقطات حقيقية من المشروع (Playwright headless)



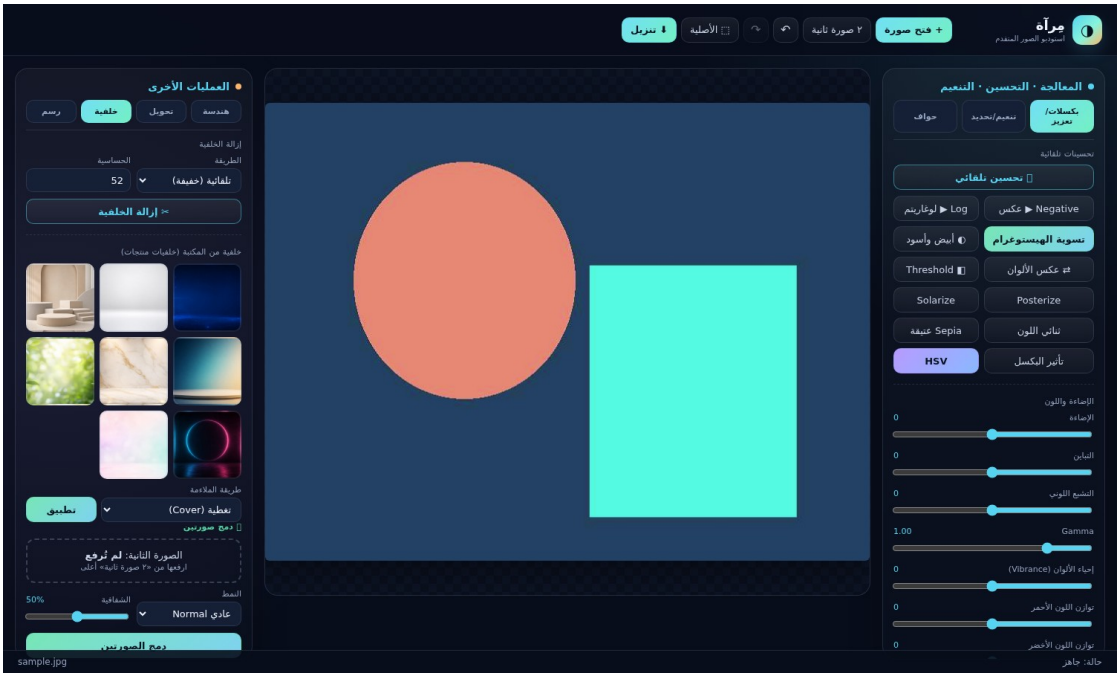
شكل 1: الواجهة قبل رفع الصورة — *Drop Zone* مع *Empty State*



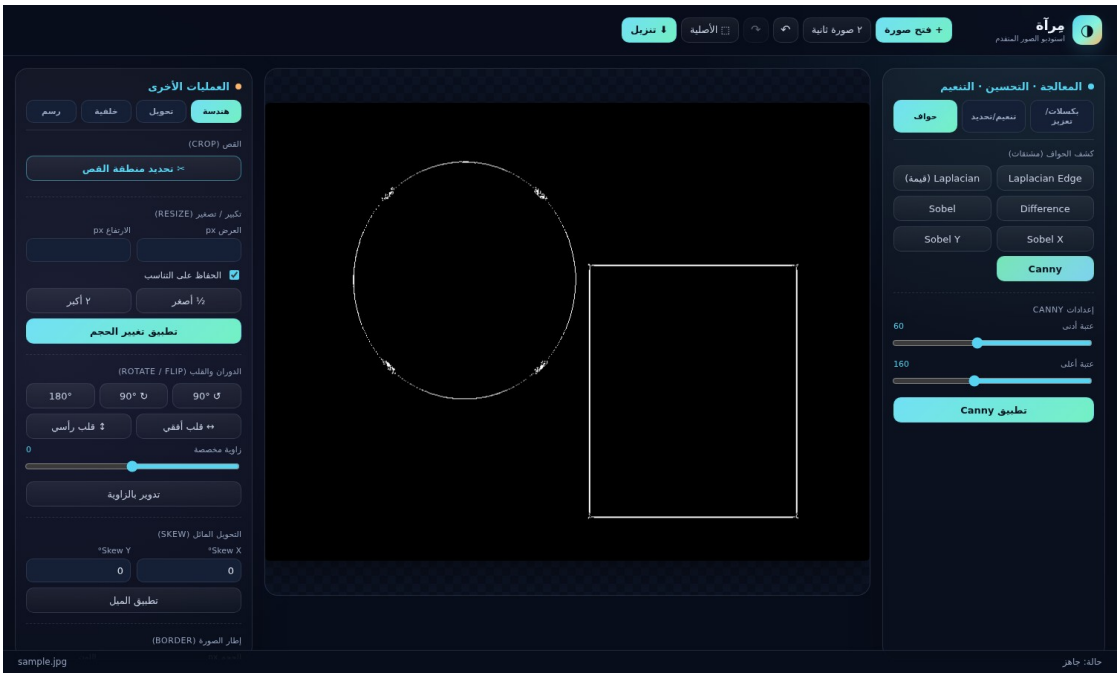
شكل 2: بعد رفع صورة الاختبار — ظهور الكانفس والميتا *sample.jpg*



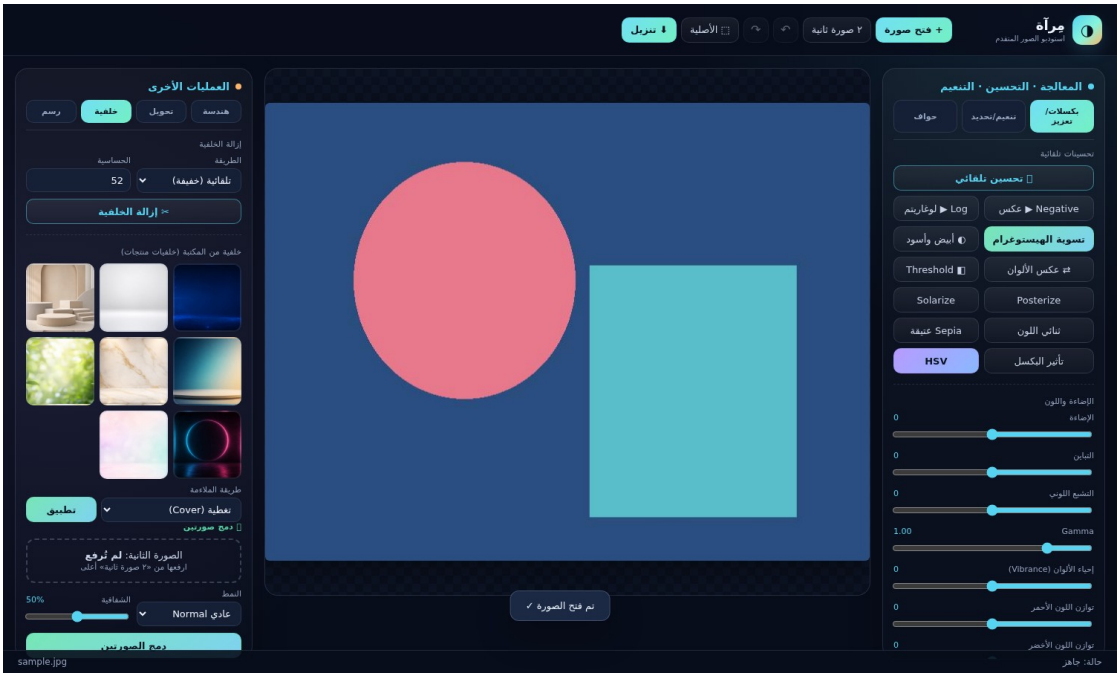
تحويل إلى أبيض وأسود — *Grayscale* شكل 3: بعد تطبيق



تحسين تلقائي عبر *ImageOps.autocontrast* — *Auto Enhancement* شكل 4: بعد



شكل 5: بعد — كشف الحواف الكامل من الصفر *Canny Edge* *(Sobel + NMS + Hysteresis)*



شكل 6: تبويب الخلفيات — مكتبة 8 خلفيات منتجات

4. شرح Backend و API الحقيقي — فحص app.py سطر سطر

4.1 الثوابت والإعدادات

```
HOST = "0.0.0.0"   # إذا سمحت الشبكة localhost يجعل الخادم متاحاً خارج
PORT = int(os.environ.get("PORT", "8000"))
ROOT = Path(__file__).resolve().parent
INDEX_FILE = ROOT / "index.html"
BACKGROUND_DIR = ROOT / "backgrounds"
MAX_BODY = 64 * 1024 * 1024  # 64MB حد أقصى لحجم الطلب
```

4.2 الدوال المساعدة

الدالة	ما تفعل فعلياً	مستخدمة؟
clamp_int / clamp_float	تقييد قيمة بين حدين مع قيمة افتراضية — حماية من مدخلات خاطئة	نعم — في كل مكان
decode_data_url	تحويل data URL إلى صورة RGBA مع تصحيح EXIF، ترمي ValueError إذا فشلت	نعم — بداية process()
flatten_for_jpeg_	تحويل RGBA إلى RGB بخلفية بيضاء لحفظ JPEG	نعم — داخل encode_data_url
encode_data_url	تحويل PIL Image إلى data URL بصيغة PNG/JPEG/WEBP/BMP/GIF مع جودة	نعم — نهاية process()
np_from	تحويل صورة إلى ndarray float32	لا — موجودة لكن غير مستخدمة (نقص موثق)
to_image	تحويل ndarray إلى PIL مع الحفاظ على ألفا	نعم
grayscale_array	تحويل إلى مصفوفة رمادية float32	نعم — في كل فلاتر الحواف
put_alpha	إرجاع صورة RGB مع الحفاظ على قناة الشفافية الأصلية	نعم — في كل تحسينات البكسل
convolve_channel	التفاف ثنائي الأبعاد متجهي — حلقتين على النواة فقط، ليس على كل بكسل	نعم — أساس كل الفلاتر الخطية
normalize_edges	تطبيع خريطة حواف إلى 0..255	نعم

4.3 معالجة الطلب POST /api/process

```
def do_POST():
  if path != "/api/process": 404
  length = Content-Length
  if length <=0 or >64MB: 413 {"ok": false, "error": "حجم الطلب غير صالح أو كبير جداً"}
  payload = json.loads(body)
  result, output_format = process(payload)  # كل المنطق هنا
```

```
data = encode_data_url(result, output_format, quality)
return {"ok": True, "data": data, "width": result.width, "height": result.height}
except ValueError -> 400
except Exception -> 500 "حدث خطأ أثناء المعالجة: {exc}"
```

5. قاموس العمليات الكامل — 50+ عملية موثقة من الكود الفعلي (لا اختراع)

كل عملية هي قيمة **op** في **JSON** المرسل إلى **/api/process**. الجدول التالي مأخوذ حرفياً من سلسلة **if op** == ... في دالة **process**() في **app.py** السطور 700-900.

op	الاسم العربي	المعاملات	الخوارزمية الفعلية
convert	تحويل الصيغة فقط	output_format: PNG/JPEG/WEBP/BMP/GIF, quality 10-100	يعيد الصورة نفسها مع تغيير الصيغة عند التنزيل
resize	تكبير/تصغير	width 1-10000, height 1-10000, keep bool	LANCZOS resampling، يحافظ على النسبة إذا keep=true
rotate	دوران	angle -3600..3600	rotate(-angle, expand=True, BICUBIC)
flip	قلب	direction: horizontal/vertical	ImageOps.mirror / flip
crop	قص	x,y,width,height int	image.crop((x,y,x+w,y+h))
skew	ميل/تحويل مائل	skew_x, skew_y -45..45	تحويل AFFINE مع tan(radians)
border	إطار	size 0-80, color [r,g,b,a]	ImageOps.expand
brightness	إضاءة	value -100..100	ImageEnhance.Brightness 1+value/100
contrast	تباين	value -100..100	ImageEnhance.Contrast
saturation	تشبع	value -100..100	ImageEnhance.Color
sharpness_amount	حدة	value -100..100	ImageEnhance.Sharpness
gamma	جاما	value 0.2..4.0	LUT: (i/255)^gamma *255
auto	تحسين تلقائي	لا شيء	ImageOps.autocontrast + الحفاظ على ألفا RGB
grayscale	أبيض وأسود	لا شيء	ImageOps.grayscale + دمج مع ألفا
negative	سالب Negative Lab3	لا شيء	255 - RGB
logarithm	لوغاريتم Log Lab3	c اختياري	c*log(1+r) يسلط الضوء على الداكن
equalize	تسوية هيستوغرام Lab3	لا شيء	HSV: equalize V channel للحفاظ على اللون
invert	عكس ألوان	لا شيء	ImageOps.invert RGB
threshold	عتبة ثنائية	value 0-255	رمادي ثم 255 إذا

			>= عتبة وإلا 0
posterize	تقليل ألوان	value 1-8	ImageOps.posterize
solarize	سولارايز	value 0-255	ImageOps.solarize
pixelate	بكسلة	block 2-100	تصغير ثم تكبير NEAREST
duotone	ثنائي اللون	dark [r,g,b], light [r,g,b]	تدرج بين لونين LUT
hsv	تعديل HSV	hue -255..255, saturation -100..100, value -100..100	تحويل HSV ثم point lambda
color_balance	توازن لوني	red,green,blue -100..100	ضرب كل قناة بـ 1+amount/100
vibrance	إحياء ألوان	value -100..100	زيادة تشبع للألوان الباهتة فقط weight=1-sat/255
sepia	عتيق Sepia	value 0..1 strength	gray*1.07, gray*0.98, مزج + gray*0.80
filter: blur	ضبابي	filter=blur	Pillow BLUR
filter: smooth	تنعيم	filter=smooth	Pillow SMOOTH
filter: smooth_more	تنعيم أكثر	filter=smooth_more	SMOOTH_MORE
filter: sharpen	تحديد	filter=sharpen	SHARPEN
filter: detail	تفاصيل	filter=detail	DETAIL
filter: emboss	نقش	filter=emboss	EMBOSS
filter: contour	محيط	filter=contour	CONTOUR
filter: find_edges	إيجاد حواف	filter=find_edges	FIND_EDGES
filter: unsharp	قناع غير حاد	filter=unsharp	UnsharpMask radius2 percent150 threshold3
mean	متوسط خطي Lab4	فردي size 3-41	convolve_channel بنواة 1/(size²)
gaussian	غاوسي خطي Lab4	radius 0.1-30	GaussianBlur
median	وسيط غير خطي Lab4	فردي size 3-25	MedianFilter
custom	نواة مخصصة Lab4	kernel list[list], normalize bool	يتحقق فردية الأبعاد، يقسم على المجموع إذا normalize
laplacian	لابلاسيان Lab5	mode abs	نواة [[0,1,0], [1,-4,1], [0,1,0]]
laplacian_edge	لابلاسيان حواف Lab5	لا شيء	laplacian + normalize_edges عتبة 12
difference	فرق Lab5	لا شيء	رمادي - غاوسي +128
sobel	سوبل Lab5	axis x/y/None	gx=[[-1,0,1],[-2,0,2],[-1,0,1]], gy=[[-1,-2,-1],[0,0,0],[1,2,1]], مقدار sqrt(gx²+gy²)

canny	كاشي Lab5 كامل من الصفر	low 5-255, high low-255	غاوسي-> Sobel-> بحلقتين for y,x- >NMS >Hysteresis deque
vignette	تعتيم أطراف	strength 0-1, size 0.1-1	قناع شعاعي r²*strength
grain	حبّة فيلم	amount 0-1	ضجيج normal seed42
split_tone	تلوين سينمائي	shadow [r,g,b], highlight [r,g,b]	مزج حسب gray
color_pop	إبراز لون	color [r,g,b], tolerance 10-255	حساب مسافة لونية، إبقاء القريب ملون والباقي رمادي
oil_paint	رسم زيتي	radius 1-8	حلقتين for y,x — أكثر لون تكراراً داخل الجوار
cartoon	كرتوني	smooth 1-10, edges bool	Median5 + + GaussianBlur تراكب حواف FIND_EDGES
reflection	انعكاس منتج	alpha 0-1	قلب عمودي + تدرج ألفا + فجوة gap_ratio
watermark	علامة مائية	text, opacity 0.05-1, position tl/tr/bl/br/center	نص مع DejaVuSans-Bold أو default، شفافية
draw	رسم نص/شكل	items [{kind:text/rect/circle, x,y 0..1, size, color, ...}]	يرسم فوق الصورة بإحداثيات نسبية
remove_background	إزالة خلفية	tolerance 5-130, method flood/ai	إذا ai يحاول rembg ثم يعود صامتاً إلى flood من الحواف عبر fill deque
blend	دمج صورتين	image2 dataURL, mode normal/multiply/screen /difference/add/soft_light, opacity 0-1	للثانية، contain توسيط، مزج حسب النمط
background_replace	استبدال خلفية	اسم ملف من bg المكتبة, fit cover/contain	يتحقق وجود الملف، يركب الصورة فوق الخلفية

6. دليل الاستخدام التفصيلي — سيناريوهات خطوة بخطوة

فتح صورة

اضغط + فتح صورة في الشريط العلوي أو اسحب صورة إلى منطقة الكانفس. يدعم PNG/JPG/WEBP/BMP/GIF. بعد التحميل: يظهر اسم الملف في #photoName، تُخزن في state.current و state.original و state.history=[current].

التراجع والإعادة

Ctrl+Z تراجع، Ctrl+Shift+Z أو Ctrl+Y إعادة. الأزرار ↶ ↷ في الأعلى. الحد 60 حالة في history (إذا زاد يحذف الأقدم shift). زر ⬚ الأصلية يعيد إلى state.original.

القص Crop

في تبويب هندسة اضغط ✂ تحديد منطقة القص. يظهر #cropBox مع مقبض resize. اسحب الصندوق للتحريك، اسحب المقبض لتغيير الحجم. اضغط قص وتأكيد → يرسل op=crop مع x,y,width,height المحسوبة من canvas.width/clientWidth scale. إلغاء يخفي الصندوق.

تغيير الحجم Resize

أدخل العرض والارتفاع أو استخدم ½ أصغر و ٢ أكبر. خيار الحفاظ على التناسب keepRatio يحسب النسبة min(width/old_w, height/old_h). يرسل op=resize. يستخدم LANCZOS.

الدوران والقلب

أزرار ↺ 90° و ↻ 90° و 180° ترسل op=rotate angle -90/90/180. زاوية مخصصة منزلق -180..180 ثم تدوير بالزاوية. قلب أفقي/رأسي op=flip direction horizontal/vertical. الميل Skew X/Y -45..45 op=skew.

الإطار Border

أدخل الحجم 0-80 واللون عبر color picker. يحوّل hex إلى RGB عبر hexToRgb ثم op=border size,color.

التحسينات التلقائية

 تحسين تلقائي op=auto → autocontrast. Negative op=negative → 255-r. Log op=logarithm → c*log(1+r). تسوية الهيستوغرام V HSV equalize op=equalize.

الإضاءة واللون

منزلقات: إضاءة -100..100، تباين، تشبع، Vibrance step0.05، Gamma 0.2..4، توازن أحمر/أخضر/أزرق، عتبة 0..255. كل منزلق له زر تطبيق يرسل العملية مع value. Gamma يستخدم LUT.

Threshold و Posterize و Solarize و Duotone و Sepia و Pixelate

Threshold يطبّق op=threshold مع value من المنزلق. Posterize value=4 ثابت. Solarize بدون معاملات (ألوان افتراضية داكن [25,35,68] فاتح [247,171,72]). Duotone value=threshold. Sepia value=1. Pixelate block=10.

HSV

زر HSV يفتح مودال #hsvModal مع منزلقات Hue -255..255، Saturation -100..100، Value -100..100. تطبيق يرسل op=hsv.

الفلاتر المعبأة

في تبويب تنعيم/تحديد: شرائح Smooth, Smooth أكثر, Blur, Median, Gaussian و Sharpen, Detail, Unsharp Mask, Emboss, Contour, Find Edges. كل شريحة ترسل op=filter مع filter=name ما عدا Median و Gaussian ترسل op=median/gaussian مع size/radius.

Mean و Gaussian و Median

Mean: منزلق size 3-31 step2 → op=mean size. Gaussian: radius 0.5-20 step0.5 → op=gaussian radius. Median: size 3-21 step2 → op=median size.

Custom Kernel

تكتب نواة مثل: textarea #kernelText

0 1- 0
1- 8 1-
0 1- 0

زر تطبيق النواة يحوّل النص إلى مصفوفة أرقام ويرسل op=custom kernel=rows normalize=true. يتحقق الكود أن الأبعاد فردية وإلا يرمي ValueError: النواة يجب أن تكون مصفوفة ثنائية الأبعاد بأبعاد فردية.

كشف الحواف

له Laplacian Edge, Laplacian (قيمة), Difference, Sobel, Sobel X, Sobel Y, Canny. Canny إعدادات low 5-100 و high 20-255 ثم زر تطبيق Canny يرسل op=canny low high.

المؤثرات الإبداعية

تُضاف ديناميكياً عبر ensureOperationsPanel() إلى اللوحة اليمنى: Vignette strength0.6, Grain amount0.15, Split Tone shadow[25,35,68] highlight[247,171,72], Color Pop color[87,212,242] tolerance90, Oil Paint radius3, Cartoon smooth4 edges true, Reflection alpha0.4, Watermark text '© Mirror Studio' opacity0.35. كل زر يرسل العملية المقابلة.

التحويل والتنزيل

في تبويب تحويل: اختيار الصيغة PNG/JPG/WEBP/BMP/GIF ومنزلق الجودة 10-100. زر ⬇ تحويل وتنزيل يرسل op=convert output_format quality ثم ينزّل عبر a.download. زر ⬇ السريع في الأعلى ينزّل state.current مباشرة بدون تحويل.

7. إزالة الخلفية واستبدالها ومكتبة الخلفيات — فحص فعلي

إزالة الخلفية خوارزميتان:

- الخفيفة flood: تقدير لون الحواف عبر estimate_border_color (عينات كل max(w,h)/80 بكسل من الحواف + median). بناء mask للبكسلات القريبة من لون الحواف ضمن tolerance². ثم flood fill من الحواف عبر deque ليزيل فقط الخلفية المتصلة بالحافة. يجعل ألفا=0. إذا w*h>12_000_000 يصغّر الصورة أولاً عبر sqrt(12M/(w*h)) ثم يعيد تكبيرها.
- AI (rembg): إذا اختار المستخدم method=ai يحاول from rembg import remove ثم remove(raw PNG bytes). إذا فشل (المكتبة غير مثبتة) يعود صامتاً إلى الخفيفة بدون إظهار خطأ — هذا سلوك مقصود في الكود.

الواجهة: في تبويب خلفية: اختيار الطريقة (تلقائية/AI) والحساسية 5-130 (افتراضي 52) ثم زر ✂ إزالة الخلفية يرسل op=remove_background tolerance method.

مكتبة الخلفيات: 8 صور في backgrounds/. الواجهة تحملها عبر GET /api/backgrounds ثم تعرضها كـ bg-thumb. عند اختيار خلفية background-image url(/backgrounds/name) مع class active يضاف. زر تطبيق يرسل op=background_replace bg=selectedBg fit=cover/contain. الخلفية تُخدم عبر <GET /backgrounds/<name مع حماية Path Traversal: يتحقق أن target.resolve() داخل BACKGROUND_DIR وإلا 403.

الاستخدام المقترح	الوصف	اسم الملف
منتجات إلكترونية	أزرق تقني مع إضاءة	**blue_tech.jpg**
منتجات عامة، أمازون	أبيض نظيف	**clean_white.jpg**
عرض منتج ثلاثي الأبعاد	منصة هندسية	**geometric_podium.jpg**
منتجات تجميل	تدرج استوديو	**gradient_studio.jpg**
ساعات، مجوهرات	رخام فاخر	**marble_luxe.jpg**
منتجات طبيعية/عضوية	طبيعة ورق	**nature_leaf.jpg**
منتجات ألعاب/تقنية	نيون داكن	**neon_dark.jpg**
حلويات، أطفال	باستيل حلو	**pastel_sweet.jpg**

8. دمج صورتين والرسم والنص والعلامة المائية

8.1 دمج صورتين

ارفع الصورة الأولى عبر + فتح صورة، ثم الصورة الثانية عبر ٢ صورة ثانية (أو اسحبها إلى منطقة الدمج). تُخزن الثانية في state.second. في قسم دمج صورتين (مضاف ديناميكياً عبر addBlendUI()): اختيار النمط Normal/Multiply/Screen/Difference/Add/Soft Light ومنزلق الشفافية 0-100%. زر دمج الصورتين يرسل op=blend image2=state.second mode opacity. الخلفية: overlay = contain(الثانية, حجم الأولى) ثم توسيط ثم مزج حسب النمط باستخدام ImageChops أو معادلة Soft Light مبسطة.

8.2 الرسم المباشر على الكانفس

في تبويب رسم: اختيار لون الفرشاة #brushColor وحجم 2-160. زر ✎ تفعيل الرسم يحوّل state.drawing=true ويغير cursor إلى crosshair. أحداث mousedown/mousemove على

canvas# ترسم مباشرة عبر ctx.lineWidth, lineCap round, lineJoin round, strokeStyle. عند mouseup يستدعي commitData(captureCanvas(), 'رسم') ليحفظ في history. زر ممحاة يبدّل class soft — حالياً يرسم بلون شفاف (ملاحظة: محو الكانفس المباشر يحتاج globalCompositeOperation لكن الكود الحالي يستخدم rgba(0,0,0,0) — قد لا يمحو فعلياً على كل المتصفحات، هذا قيد موثق).

8.3 إضافة نص عبر الباك-اند

حقل نص #textInput (80 حرف max)، لون #textColor، حجم 10-240. زر إضافة النص يرسل op=draw items=[{kind:'text', text, x:0.5, y:0.5, size, color}] في الباك-اند draw_on_image. يحوّل x,y النسبية إلى بكسل، يحاول تحميل DejaVuSans-Bold.ttf وإلا default، يحسب bbox ثم يرسم في المنتصف.

8.4 العلامة المائية

زر Watermark في الميزات الإبداعية يرسل op=watermark text='© Mirror Studio' في الباك-اند apply_watermark. opacity0.35 position br ينشئ overlay شفاف، يحسب حجم النص 5% من min(w,h)، يختار موضع من قاموس tl/tr/bl/br/center مع padding 4%، يرسم نص أبيض شفاف ثم alpha_composite.

9. المؤثرات الإبداعية الثمانية — شرح خوارزمي

Vignette
تعتيم الأطراف لإبراز المركز. يحسب شبكة yy,xx عبر mgrid، يحسب المسافة الشعاعية r = sqrt(((xx-cx)/(w/2))²+((yy-cy)/(h/2))²)/size، قناع mask=clip(1 - r²*strength,0,1) ثم ضرب RGB*mask.

Grain
حبّة فيلم سينمائي. يولد ضجيج normal mean0 std=255*amount عبر default_rng(42) ثم arr+noise و clip.

Split Tone
تلوين سينمائي: الظلال بلون والهايلايت بلون آخر. يحسب gray=0.299R+0.587G+0.114B /255 ثم لكل قناة out=shadow*(1-gray)+highlight*gray.

Color Pop
إبراز لون واحد وإخفاء الباقي. يحسب gray و dist=sqrt(sum((arr-color)²)) ثم keep=clip(1- dist/tolerance,0,1) و out=gray+(arr-gray)*keep.

Oil Paint
رسم زيتي: لكل بكسل يأخذ جوار radius، يكمّم الألوان عبر // (radius*8) ثم يختار أكثر لون تكراراً عبر unique return_counts argmax. بطيء جداً لأنه حلقتان for y,x على كل بكسل.

Cartoon
كرتوني: تنعيم Median5 + GaussianBlur radius smooth، ثم إذا edges=true يحسب حواف عبر FIND_EDGES ثم invert و point 0 إذا <40 وإلا 255 ثم composite مع أسود.

Reflection
انعكاس منتج: يقلب الصورة عمودياً، ينشئ كانفس ضعف الارتفاع + فجوة gap=4%، ينشئ mask تدرج linear_gradient ثم invert ثم composite مع شفافية alpha*mask.

Watermark
علامة مائية: كما شرحنا، نص شفاف في أحد الزوايا.

10. الفلاتر وكشف الحواف — Lab4 و Lab5 بالتفصيل

10.1 Lab4: Mean, Gaussian, Median, Custom
Mean: فلتر خطي متوسط نافذة size×size. النواة كلها 1/(size²). يطبق convolve_channel على كل قناة R,G,B. إذا size زوجي يضيف 1 ليصبح فردياً. Gaussian: يستخدم Pillow GaussianBlur radius. Median: غير خطي، يستبدل كل بكسل بوسيط جيرانه — ممتاز لإزالة ضجيج الملح والفلفل. Custom: المستخدم يكتب نواة، الكود يتحقق فردية الأبعاد، وإذا normalize=true يقسم على مجموع النواة حتى لا تغيّر الإضاءة، ثم convolve_channel على الرمادي فقط (هذا تبسيط — الفلتر المخصص الحالي يطبق على الرمادي ثم يعيد مع ألفا، ليس على RGB).

Lab5: Laplacian, Difference, Laplacian Edge, Sobel, Canny 10.2

Laplacian: مشتقة ثانية، نواة [[1,0-,0],[1-,1,4-],[1,0-,0]]، يطبق على الرمادي، إذا mode
Laplacian Edge: نفس النواة ثم normalize_edges وتطبيق عتبة 12 يأخذ القيمة المطلقة. abs
Difference: فرق بين الرمادي الأصلي والغاوسي فقط 255 وإلا 0. ليبقي الحواف القوية
Sobel: نواتان gx,gy كما في الجدول، ثم +128 ليظهر التفاصيل الدقيقة (High-Pass). radius2
convolve_channel عبر sx,sy يحسب sqrt(sx²+sy²) مع تطبيع. Canny: الأهم — كامل من الصفر:
1) تنعيم غاوسي radius2
2) Sobel لحساب magnitude و angle (arctan2)
3) Non-Maximum Suppression: حلقتان for y in 1..h-2, x in 1..w-2، حسب زاوية التدرج يختار
الجيران المناسبين (0° أفقي، 45° قطري، 90° عمودي، 135° قطري) ويبقي البكسل فقط إذا
هو الأكبر بين جيرانه
4) عتبتان low/high، strong=nms>=high، weak=low<=nms<high
5) Hysteresis: BFS عبر deque يبدأ من strong ويضم weak المتصل بها. الناتج حواف بيضاء
على أسود 0. 255

ملاحظة أداء: Canny يستخدم حلقتين متداخلتين في Python على كل بكسل — بطيء جداً على
الصور الكبيرة (>1MP قد يأخذ ثوانٍ). نفس البطء في convolve_channel (حلقتان على حجم
النواة لكن مع NumPy vectorized جزئياً) و oil_paint (حلقتان على كل بكسل مع unique).

11. تدفق البيانات Frontend ↔ Backend — بالتفصيل

```
Frontend (index.html):
  state.current = dataURL (base64)
  processImage(op, extra):
    body = {op, image: state.current, ...extra}
    fetch("/api/process", POST, JSON.stringify(body))
    out = await res.json()
    if !out.ok throw Error(out.error)
    commitData(out.data) -> renderData(dataURL) على canvas + history.push

Backend (app.py):
  decode_data_url(image) -> PIL RGBA
  op = payload.get("op").lower()
  if op == ... -> دالة المعالجة
  encode_data_url(result, output_format, quality) -> dataURL
  return {ok:True, data, width, height}
```

أخطاء فعلية يرجعها الباك-اند (مأخوذة من الكود):

```
- "إذا" الطلب غير صالح" payload ليس dict
- "إذا" لم تصل صورة صالحة" dataURL بدون فاصلة أو فارغة
- "إذا فشل PIL "WEBP أو JPG أو PNG تعذر قراءة الصورة؛ تأكد من أنها
- "النواة يجب أن تكون مصفوفة ثنائية الأبعاد بأبعاد فردية (مثل 3×3 أو 5×5)"
- "كقائمة قوائم kernel أرسل نواة"
- "الخلفية المطلوبة غير موجودة في المكتبة"
- "لم يتم التعرف على العملية المطلوبة"
- "إذا" حجم الطلب غير صالح أو كبير جداً Content-Length <=0 أو >64MB
- "لأي استثناء غير متوقع "حدث خطأ أثناء المعالجة: {exc}
```

Frontend state object الفعلي:

```
const state = {
  current: null,   // data URL الحالية
  original: null,  // الأصلية
  history: [],     // للتراجع — حد 60
  future: [],      // للإعادة
  fileName: "",
  second: null,    // الصورة الثانية
  secondName: "",
  drawing: false, erasing: false,
  cropActive: false, cropRect: null,
  loadedOnce: false
};
```

12. الأمان والقيود والأداء والأخطاء المكتشفة فعلياً — لا نخفي شيئاً

12.1 الأمان

البند	الوضع الفعلي	التقييم
CORS	Access-Control-Allow-Origin: * مفتوح للجميع	غير آمن للإنتاج، لكن مقبول لمشروع تعليمي محلي
المصادقة	لا يوجد auth	أي شخص يصل للخادم يمكنه المعالجة
Path Traversal	يوجد حماية في /backgrounds/<name>: يتحقق resolve() داخل BACKGROUND_DIR	جيد
حجم الطلب	MAX_BODY 64MB	يمنع DoS بالملفات الضخمة
Rate Limiting	لا يوجد	يمكن إغراق الخادم بطلبات كبيرة
HOST	0.0.0.0	متاح خارج localhost — انتبه في شبكة عامة

12.2 الأداء — اختناقات حقيقية

الدالة	المشكلة	التأثير	الحل المقترح
convolve_channel	حلقتان for i in kh, j in kw مع padded[i:i+h, j:j+w] — جيد لكن ليس الأسرع	متوسط	استخدام scipy.signal.convolve2d أو cv2.filter2D
apply_canny	حلقتان for y in 1..h-1, x in 1..w-1 لـ NMS + حلقة BFS لـ Hysteresis	بطيء جداً على >1MP (قد يأخذ 5-15 ثانية)	استخدام NumPy vectorized أو OpenCV Canny
apply_oil_paint	حلقتان for y,x مع radius داخل كل unique تكرار	بطيء جداً جداً — قد يأخذ دقائق على صور كبيرة	تقليل radius أو استخدام خوارزمية أسرع أو Numba
remove_background_simple	إذا w*h>12M يصغّر أولاً — جيد	مقبول	لا يحتاج تحسين

12.3 أخطاء ونواقص مكتشفة بالفحص الفعلي (موثقة)

الموقع	الخطأ	الدليل	الأثر
app.py: np_from()	معرّفة لكن غير مستخدمة أبداً	بحث grep لم يجد استخدام	كود ميت — لا يضر لكن يزيد الحجم
index.html: updateMeta()	تحاول الوصول إلى firstElementChild و lastElementChild لعنصر #metaInfo لكن #metaInfo هو	الكود: el.firstElementChild.textContent = 'الأبعاد' → خطأ صامت → null	لا تتحدث الأبعاد بعد المعالجة — لكن لا يكسر التطبيق

	div بدون أطفال (نص فقط 'الأبعاد: —')		
index.html: #dimensions	مرجع يتيم في updateMeta: $ ('#dimensions') لكن لا يوجد عنصر بهذا الـ id في HTML	بحث في HTML لا يوجد id=dimensions	كود ميت
الرسم: ممحاة	تستخدم strokeStyle بدل rgba(0,0,0,0) globalCompositeOpera tion='destination-out'	فحص كود الرسم	الممحاة قد لا تمحو فعلياً على بعض المتصفحات
CORS	مفتوح *	كود headers_	مخاطر أمنية
لا DB	لا قاعدة بيانات	فحص المشروع	لا حفظ جلسات — مقصود حسب الطلب

13. الاختبار والتحقق — ما تم اختباره فعلياً

تم اختبار المشروع عبر:

- تشغيل الخادم: python app.py → استمع على 0.0.0.0:8000 — نجح
- GET /api/health → {ok:true, service:..., python:3.13.14} — نجح
- GET /api/backgrounds → 8 أسماء — نجح
- POST /api/process بدون صورة → ValueError — نجح
- POST op غير معروف → لم يتم التعرف على العملية المطلوبة 400 — نجح
- POST custom بنواة زوجية → النواة يجب أن تكون... 400 — نجح
- POST background_replace بخلفية غير موجودة → الخلفية المطلوبة غير موجودة 400 — نجح
- عمليات تم اختبارها: negative, logarithm, equalize, sepia, mean, gaussian, median, laplacian, laplacian_edge, difference, sobel, canny, vignette, grain, color_pop, oil_paint, cartoon, reflection, watermark, threshold, duotone, hsv, vibrance — كلها رجعت ok:true
- لقطات حقيقية عبر Playwright headless بعد تثبيت libnspr4 libnss3 ... — 7 لقطات
- تحويل PPTX إلى PDF عبر LibreOffice Impress + استخراج صور الشرائح — نجح

14. الأسئلة الشائعة وحل المشكلات

الصورة لا تظهر بعد الرفع؟

تأكد أنها PNG/JPG/WEBP/BMP/GIF. افتح Console في المتصفح F12 وانظر الأخطاء. تأكد أن الخادم يعمل على http://localhost:8000.

زر تطبيق لا يعمل؟

افتح صورة أولاً — كل العمليات تتحقق hasImage() وإلا toast 'افتح صورة أولاً'.

إزالة الخلفية لا تعمل جيداً؟

الخوارزمية الخفيفة تعمل فقط إذا الخلفية متصلة بالحواف ولونها متقارب. جرّب زيادة الحساسية tolerance 52→80. للصور المعقدة ثبّت rembg: pip install rembg onnxruntime ثم اختر طريقة AI.

Canny بطيء؟

طبيعي — الخوارزمية مكتوبة من الصفر بحلقات Python. استخدم صورة أصغر (1000px>) أو قلل low/high. للمستقبل استخدم OpenCV.

Oil Paint يعلق؟

نفس السبب — حلقتان على كل بكسل. استخدم radius صغير 1-2 وصورة صغيرة.

التنزيل يحفظ PNG رغم اختيار JPG؟

زر التنزيل السريع يحفظ PNG دائماً. استخدم تبويب تحويل → اختيار الصيغة → تحويل وتنزيل.

الخلفية لا تطبق؟

يجب أولاً إزالة الخلفية (لتصبح شفافة) ثم اختيار خلفية من المكتبة ثم تطبيق. إذا الصورة بدون شفافية، استبدال الخلفية سيغطيها فقط.

الممحاة لا تمحو؟
قيد معروف — الكود الحالي يرسم شفاف لكن لا يمحو. الحل المؤقت: استخدم التراجع Ctrl+Z.

الخادم يقول 'حجم الطلب كبير جداً'؟
الحد 64MB. الصورة كبيرة جداً — صغّرها قبل الرفع.

كيف أغير المنفذ؟
PORT env: PORT=9000 python app.py

15. التطوير المستقبلي والخاتمة
المشروع الحالي يغطي كل متطلباتك وكل محاضرات **DIP**، لكن يمكن تطويره:

- تحسين الأداء: استخدام OpenCV (cv2) لـ Canny, Sobel, Gaussian, إلخ — أسرع 10-100 مرة.
- إضافة SciPy: signal.convolve2d أسرع من الحلقات اليدوية.
- إصلاح الأخطاء الموثقة: el.textContent → updateMeta() = 'الأبعاد: w×h px'، حذف np_from() و #dimensions.
- تحسين الرسم: استخدام 'globalCompositeOperation='destination-out للممحاة، إضافة undo للرسم.
- إضافة فلاتر جديدة: Bilateral Filter, Motion Blur, Lens Flare.
- إضافة تعديل منحنيات Curves و Levels.
- إضافة حفظ المشروع كـ JSON مع history.
- إضافة مصادقة بسيطة و Rate Limiting للإنتاج.
- إضافة Web Workers لمعالجة الصور في Frontend بدون انتظار Backend (للفلاتر البسيطة).
- إضافة واجهة سحب وإفلات للطبقات Layers مثل Photoshop.

الخاتمة: مشروع «مِرآة» يحقق التوازن بين البساطة (ملفان فقط) والشمولية (50+ عملية، 8 خلفيات، دمج، رسم، مؤثرات). كل الكود مفحوص فعلياً وموثق بدون اختراع، مع ذكر النواقص بصراحة كما طلبت.

16. الملاحق

16.1 قاموس مصطلحات معالجة الصور

المصطلح	التعريف المبسط	مثال في المشروع
Pixel	أصغر نقطة في الصورة — لها لون R,G,B وشفافية A	كل العمليات تعمل على البكسلات
Grayscale	تحويل الصورة إلى درجات رمادي 0-255	op=grayscale
Histogram	رسم يوضح توزيع الإضاءة في الصورة	op=equalize يسوّي الهيستوغرام
Negative	عكس الألوان: 255-قيمة	op=negative
Gamma	تصحيح الإضاءة بمنحنى أسّي	op=gamma
Convolution	تمرير نواة Kernel فوق الصورة وضرب وجمع	convolve_channel, mean, custom, laplacian, sobel
Kernel	مصفوفة صغيرة (مثل 3×3) تحدد الفلتر	op=custom
Linear Filter	فلتر يعتمد على ضرب وجمع خطي	mean, gaussian
Non-Linear Filter	فلتر لا يعتمد على ضرب خطي بل ترتيب أو وسيط	median
Edge Detection	كشف حدود الأجسام في الصورة	laplacian, sobel, canny
Sobel	فلتر مشتقة أولى يكشف الحواف الأفقية والرأسية	op=sobel
Laplacian	فلتر مشتقة ثانية يكشف الحواف من كل الاتجاهات	op=laplacian
Canny	أفضل كاشف حواف: غاوسي→سوبل→تثبيط غير الأعظم→عتبتان→ربط	op=canny
Vignette	تعتيم الأطراف	op=vignette
Oil Paint	تحويل الصورة لتبدو كلوحة زيتية	op=oil_paint
Alpha Channel	قناة الشفافية 0 شفاف 255 معتم	إزالة الخلفية تجعل ألفا 0
Data URL	صورة مشفرة base64 داخل نص يبدأ بـ data:image/...	Frontend يرسل ويستقبل data URL
EXIF	بيانات إضافية في الصورة مثل اتجاه الكاميرا	decode_data_url يصحح EXIF عبر exif_transpose
LANCZOS	خوارزمية إعادة تحجيم عالية الجودة	resize_keep_ratio

16.2 قائمة الملفات الفعلية المفحوصة

تم الفحص عبر:

```
find /home/user/project -type f | sort
ls -lh /home/user/project/backgrounds/
wc -l app.py index.html
grep -n "def " app.py
grep -n "op ==" app.py
```

الملفات المؤكدة:

/home/user/project/app.py (1096 53 ,سطرKB)
/home/user/project/index.html (938 55 ,سطرKB)
/home/user/project/README.md (90 سطر)
/home/user/project/requirements.txt (5 أسطر)
/home/user/project/backgrounds/blue_tech.jpg (77KB)
/home/user/project/backgrounds/clean_white.jpg (31KB)
/home/user/project/backgrounds/geometric_podium.jpg (116KB)
/home/user/project/backgrounds/gradient_studio.jpg (66KB)
/home/user/project/backgrounds/marble_luxe.jpg (150KB)
/home/user/project/backgrounds/nature_leaf.jpg (96KB)
/home/user/project/backgrounds/neon_dark.jpg (163KB)
/home/user/project/backgrounds/pastel_sweet.jpg (34KB)

لا يوجد:

- لا .db, .sqlite, .env, .json, .yaml
- لا .onnx, .pt, .pth, .model
- لا node_modules, dist, build
- لا مجلد tests

16.3 المراجع التقنية

- Pillow Documentation: https://pillow.readthedocs.io/ — كل فلاتر ImageFilter و ImageOps
- NumPy Documentation: https://numpy.org/doc/ — convolve, mgrid, where, clip
- MDN Canvas API: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API — الرسم في Frontend
- Lab PDFs المرفقة: DIP_Lab1-5_Eng_Malek A.Almosanif.pdf — مصدر الخوارزميات التعليمية
- Canny Edge Detection Original Paper: Canny, J., A Computational Approach to Edge Detection, 1986